

بسمه تعالی

# پروژه DDQN

عنوان پروژه: آموزش یک عامل بازیگر هوشمند با استفاده از Double DQN و شبکه کانولوشنی (CNN)

هدف پروژه

در این پروژه، دانشجویان یک عامل هوشمند (Agent) را طراحی و پیاده‌سازی می‌کنند که بتواند به صورت خودکار یک بازی ویدیویی ساده (مانند CartPole, Breakout یا Pong) را از طریق مشاهده تصاویر اولیه (Pixel Input) یاد بگیرد و بازی کند.

در این راستا، از ترکیب دو فناوری پیشرفته استفاده می‌شود:

شبکه کانولوشنی (CNN) برای استخراج ویژگی‌های تصویری از فریم‌های بازی.

Double Deep QNetwork (DDQN) برای بهبود پایداری و دقت یادگیری تقویتی.

این پروژه به دانشجویان کمک می‌کند تا با رویکردهای پیشرفته یادگیری عمیق تقویتی (Deep Reinforcement Learning) آشنا شوند و تجربه عملی در پیاده‌سازی مدل‌های هوش مصنوعی مشابه سیستم‌های واقعی (مثل عامل‌های بازی DeepMind) کسب کنند.

وظایف اصلی دانشجو

1. درک محیط بازی و پیش‌پردازش تصویر:

استفاده از کتابخانه `gym` یا `gymnasium` برای دسترسی به محیط‌های بازی (مثل `Pongv4` یا `Breakoutv4`).

پیش‌پردازش تصاویر: تغییر اندازه، تبدیل به خاکستری، نرمال‌سازی و ایجاد "استک فریم" (Frame Stacking)

2. طراحی شبکه عصبی کانولوشنی (CNN):

پیاده سازی یک شبکه CNN ساده با لایه های کانولوشنی، Pooling و Fully Connected.

خروجی شبکه Qvalues: برای هر عمل ممکن در بازی.

3. پیاده سازی الگوریتم Double DQN:

تفاوت DDQN با DQN: جداسازی انتخاب عمل و ارزیابی Qvalue برای کاهش اغراق در Qvalue.

استفاده از دو شبکه: شبکه ارزیابی (Online) و شبکه هدف (Target).

به روز رسانی دوره ای شبکه هدف (Target Network Update).

4. اجرای چرخه یادگیری:

جمع آوری تجربه در یک حافظه تجربه (Replay Buffer).

نمونه گیری تصادفی (Minibatch) برای آموزش شبکه.

آموزش شبکه با استفاده از تابع خطا (MSE) یا Huber Loss.

5. تحلیل و ارائه نتایج:

رسم نمودارهایی از روند یادگیری: بازده کلی (Total Reward)، تعداد اقدامات، زمان یادگیری.

مقایسه عملکرد DDQN با DQN پایه (در صورت امکان).

ضبط یا نمایش ویدئویی از عملکرد عامل در انتهای آموزش.

مفاهیم آموزشی کلیدی

یادگیری تقویتی عمیق (Deep Reinforcement Learning)

شبکه‌های کانولوشنی (CNN) برای پردازش تصویر

الگوریتم Double DQN و مزایای آن نسبت به DQN

تجربه گرا (Experience Replay) و شبکه هدف (Target Network)

پیش‌پردازش تصویر و Frame Stacking

تعادل بین اکتشاف و بهره‌برداری ( $\epsilon$ greedy)

ابزارها و کتابخانه‌های مورد نیاز

زبان Python :

کتابخانه‌ها:

`gymnasium` (یا `gym` برای محیط بازی)

`torch` (یا `tensorflow/keras` برای شبکه عصبی)

(`numpy`, `matplotlib`, `opencvpython` اختیاری برای پیش‌پردازش)

(اختیاری `pygame` (یا `tensorboard` برای نمایش پیشرفت

خروجی‌های مورد انتظار

1. کد پایتون کامل و مستند شده با ساختار منظم:

`agent.py` کلاس عامل و شبکه DDQN

`environment.py` مدیریت محیط و پیش‌پردازش

`train.py` لوپ اصلی آموزش

2. گزارش نهایی شامل:

توضیح معماری شبکه

نمودارهای یادگیری

تصاویر/ویدئو از عملکرد عامل

تحلیل نقاط قوت و ضعف

3. (اختیاری) مقایسه DDQN با DQN و تحلیل بهبود عملکرد

💡 چرا این پروژه مهم است؟

این پروژه مستقیماً از مقاله معروف (DeepMind (2015 در مورد بازی‌های Atari با DQN الهام گرفته شده است. با انجام این پروژه، دانشجویان:

تجربه عملی در پیاده‌سازی یک سیستم هوش مصنوعی پیشرفته کسب می‌کنند.

با چالش‌های واقعی یادگیری تقویتی (مثل پایداری، ناهمگونی داده) آشنا می‌شوند.

مهارت‌های مورد نیاز برای ورود به حوزه‌های پیشرفته مانند رباتیک، خودروهای خودران یا هوش مصنوعی بازی را تقویت می‌کنند.